SIMATS ENGINEERING

Department of Information Technology

DSA01 – Object-Oriented Programming with C++

ASSIGNMENT 1 – CO1

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs – the object-oriented paradigm and its basic principles, tokens, keywords, identifiers, constants, data types, type compatibility, variables, operators, implicit conversions, operator precedence and control structures.

Student Name	I. Lahari	**Register Number**	192472183
Department / Section	CSE-AI	**Academic Year / Semester**	3 year
Faculty Name	Amirtha. S	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	10 -11 - 2026	**Date of Evaluation**	
GitHub Repository	DSA01-<RegisterNumber>-Assignment1	**Compiler**	g++ 13.3.0, -std=c++17 -Wall -Wextra

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Object-Oriented Analysis	4		
2	Tokens, Keywords, Identifiers and Constants	3		
3	Data Types and Compatibility	4		
4	Implicit Conversions	4		
5	Operators and Precedence	4		
6	Control Structures	4		
7	Procedural vs Object-Oriented	4		
8	Input and Output	4		

Task	Task Title	Max	Marks Awarded	Remarks
9	Implementation and Testing	4		
10	Expectation vs Execution	4		
11	Final Recommendation	4		
12A	Re-evaluate: Impact Analysis	2		
12B	Re-evaluate: Evaluation of Original Design	2		
12C	Re-evaluate: Adaptation or Redesign	2		
12D	Re-evaluate: Evidence-Based Defence	2		
	TOTAL	50		Percentage:

OVERALL REMARKS

I. lahari		
Student Signature	**Faculty Signature**	**Program Director**

OBJECT-ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 1 – CO1

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs – the object-oriented paradigm and its basic principles, tokens, keywords, identifiers, constants, data types, type compatibility, variables, operators, implicit conversions, operator precedence and control structures.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: 10-11-2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Assignment Title: Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System (Total Marks - 50)

Mandatory Execution and GitHub Submission

- Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
- Create a GitHub repository named DSA01-<RegisterNumber>-Assignment1 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (Task01, Task02 …), containing the C++ source files, input files and output screenshots.
- Include a README.md with your name, register number, compiler and version, the exact g++ command used to compile, the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. A single upload on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report.

GitHub Repository Link: https://github.com/lahari1426/DSA0115_192472183.git

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Important Submission and Evidence Note

This report is a complete draft aligned with the 12 tasks in the supplied assignment brief. Replace the student details and GitHub URL before submission. The assignment specifically requires each student to compile and execute every program, save source files and output screenshots in separate task folders, make progressive commits, and submit a working GitHub repository.

The expected outputs and sample test results below are derived from the stated C++ rules and the supplied scenario. They are not a claim that the programs were executed on the student's computer. Before final submission, compile and run each program using the compiler/version installed on your system, record the actual sizeof and compiler warnings, attach screenshots, and update the evidence fields with your own observations.

- Suggested compile command: g++ -std=c++17 -Wall -Wextra -pedantic filename.cpp -o program
- Suggested repository: DSA01-<RegisterNumber>-Assignment1
- Create folders Task01 through Task12. Store each task's .cpp file, input (if any), output screenshot and notes in its folder.
- Commit progressively: at least one meaningful commit for each task.
- README.md must include name, register number, compiler/version, compile command, run steps and a short result for every task.

Scenario and Design Assumptions

The metro has Inner, Middle and Outer zones with base fares of Rs 10, Rs 20 and Rs 30. Peak hours are 8:00–10:00 AM and 5:00–8:00 PM. The peak surcharge is 20%. A journey is permitted only when the balance after deduction is at least Rs 50. Student and senior-citizen concessions are mentioned in the scenario, but the brief does not specify their percentages; therefore, the original implementation below does not invent a concession rate. A configurable concession rule is recommended for a later approved fare policy.

For boundary interpretation, peak intervals are treated as start-inclusive and end-exclusive: 08:00 ≤ time < 10:00 and 17:00 ≤ time < 20:00. Therefore, exactly 10:00 AM and exactly 8:00 PM are off-peak.

TASK 1 – Understand the Problem and Identify the Objects (4 Marks)

Object-oriented analysis

Object	Data (attributes)	Behaviour (methods)	Reason for the object
SmartCard	cardId, balancePaise, passengerId	canTravel(fare), deduct(fare), recharge(amount)	Owns the stored balance and protects the minimum-balance rule.
Passenger	passengerId, category (regular/student/senior)	getConcessionCategory()	Keeps passenger identity and concession eligibility separate from card balance.
Journey	originZone, farthestZone,	isValid(), calculateFare()	Represents one travel transaction and its

	entryTime, date		travel conditions.
Zone	zone identifier / enum value	—	Represents the fare zone selected for the journey.
FareTable	base fares, surcharge rate, minimum balance	baseFare(zone), finalFare(zone, time, policy)	Centralises fare policy so values are not repeated throughout the program.

In a procedural solution, the balance, fare values and time rules would be stored in separate variables or arrays, and free functions would calculate fares and deduct money. As the program grows, several functions may directly modify the same balance. In an object-oriented solution, SmartCard, Journey and FareTable group related data with the operations that use them. The SmartCard class can prevent invalid balance changes, while FareTable provides one place for fare policy.

OOP principles in this system

- Encapsulation: SmartCard keeps balancePaise private. The balance changes only through validated methods such as deduct() and recharge().
- Abstraction: The gate calls calculateFare() and canTravel() without needing to know every internal arithmetic step.
- Inheritance: StudentPassenger and SeniorPassenger may derive from Passenger if they have additional concession rules. This is useful only when the categories need different behaviour.
- Polymorphism: A virtual concessionRate() or calculateConcession() method can be overridden by passenger categories. The rates must come from an approved fare policy.

Journey rules: (1) the zone must be valid; (2) the hour and minute must be valid; (3) fare must not be negative; (4) the card balance must be at least fare + Rs 50 before travel; (5) a denied journey must not change the balance; (6) all money arithmetic must use integer paise; and (7) peak boundaries must be consistent.

Alternative: procedural design and complexity

A procedural design can use long long balancePaise, a fare array and functions such as calculateFare(zone, peak), canTravel(balance, fare) and deduct(balance, fare). Both procedural and object-oriented versions take O(1) time per journey and O(1) memory per card. The important difference is maintenance: a fare rule change may require editing several procedural functions or shared variables, whereas a central FareTable/class method limits the change to one policy location. OOP does not automatically make execution faster; it improves organisation and control of valid state.

Evidence

Attach a UML class diagram showing SmartCard, Passenger, Journey and FareTable, their attributes, methods and relationships. A suitable relationship is: Passenger owns/uses a SmartCard; Journey refers to a SmartCard and FareTable; FareTable provides fare policy.

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly (3 Marks)

Tokens

Token category	Examples from the program
Keywords	class, private, public, enum, constexpr, if, else, return, switch
Identifiers	SmartCard, balancePaise, BASE_PAISE, calculateFare
Constants	10, 20, 30, 1000, MIN_BALANCE_PAISE
Operators	+ , -, *, /, %, ==, >=, &&, ||, -=, ?:
Punctuators	{ }, ( ), [ ], ;, ,, .

Constants and zone representation

A #define macro performs textual substitution and does not provide normal C++ type checking or scope. A const variable is read-only after initialisation. A constexpr value is intended to be usable in compile-time expressions and is preferred for fixed fare policy values. enum class gives zones a scoped type and prevents accidental mixing with unrelated integers.

```
#include <array>
#include <cstdint>

enum class Zone { Inner, Middle, Outer };

constexpr std::array<long long, 3> BASE_PAISE{
   1000LL, 2000LL, 3000LL
};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000LL;
```

Identifier conventions: use PascalCase for class and enum names (SmartCard, Zone), UPPER_SNAKE_CASE for named constants (MIN_BALANCE_PAISE), and lowerCamelCase for variables and functions (balancePaise, calculateFare). Names should describe meaning and avoid reserved keywords.

Alternative: #define versus constexpr and enum class

With #define, a fare such as #define OUTER_FARE 3000 is replaced by the preprocessor before compilation. It has no ordinary object storage, but it also has no type or namespace scope. constexpr long long OUTER_FARE = 3000LL is type-checked and scoped. enum class Zone is safer than plain integer zone codes because an arbitrary int cannot be passed as a Zone without an explicit conversion. Neither approach materially changes O(1) fare lookup time. A constexpr value commonly requires no run-time storage when used as a compile-time constant.

TASK 3 – Choose Data Types and Check Type Compatibility (4 Marks)

Value	Recommended type	Reason
Money / fare / card balance	long long paise	Exact integer arithmetic; avoids binary floating-point rounding.
Peak flag	bool	Only true or false.
Zone	enum class Zone	Type-safe named values.
Hour / minute	int	Small whole-number values with easy range validation.
Passenger category	enum class PassengerType	Named and type-safe categories.
Journey count	long long or std::uint64_t	Supports large network totals.
Single character input	char	Stores one character.
Calculated ratio for analysis	double	Suitable for non-currency measurements.
Inferred local variable	auto	Useful when the initialiser makes the type clear.

Money is stored as paise: Rs 10.00 is 1000 paise, Rs 30.00 is 3000 paise and Rs 50.00 is 5000 paise. Integer addition and subtraction are exact while values remain in range. A double stores an approximation for many decimal fractions, so repeated additions can produce a small representation error.

Type compatibility: when int and double occur in one arithmetic expression, the int is normally converted to double. When an integer is used as a bool, zero becomes false and non-zero becomes true. Integer division discards the fractional part. Signed/unsigned comparisons can convert a negative signed value to a large unsigned value, producing surprising results.

Size and range evidence

The exact sizeof values depend on the compiler and platform. Record the results of this program on the system used for submission:

```
#include <iostream>
#include <climits>
int main() {
    std::cout << "sizeof(bool) = " << sizeof(bool) << '\\n';
    std::cout << "sizeof(char) = " << sizeof(char) << '\\n';
    std::cout << "sizeof(int) = " << sizeof(int) << '\\n';
    std::cout << "sizeof(long long) = " << sizeof(long long) << '\\n';
    std::cout << "sizeof(float) = " << sizeof(float) << '\\n';
    std::cout << "sizeof(double) = " << sizeof(double) << '\\n';
    std::cout << "INT_MAX = " << INT_MAX << '\\n';
    std::cout << "LLONG_MAX = " << LLONG_MAX << '\\n';
```

```
}
```

On a common modern GCC platform, int is 4 bytes and long long is 8 bytes, but the program output is the evidence to report. The maximum one-day collection at 3,000,000 journeys, even if every journey costs Rs 36, is Rs 108,000,000 (10,800,000,000 paise). This exceeds the usual 32-bit signed int range in paise. A 64-bit long long is appropriate.

Alternative: double rupees

Both double and long long commonly occupy 8 bytes, and one addition is O(1). However, double represents many decimal fractions approximately. For example, repeated addition of 0.1 may not produce an exact decimal result. An experiment can compare n additions of 0.1 using double with n additions of 10 integer paise; report the observed difference for n = 20,000, 500,000 and 3,000,000. Integer paise is the safer representation for a fare ledger.

TASK 4 – Control Implicit Conversions (4 Marks)

Important conversion points include: int division before assignment to double; integer-to-double conversion when one operand is double; char promotion to int in arithmetic; non-zero int conversion to bool; and signed-to-unsigned conversion in mixed comparisons. Use static_cast when a conversion is intentional and brace initialisation to reject narrowing conversions.

```
int a = 7, b = 2;
double wrong = a / b;                    // 3.0: integer division first
double correct = static_cast<double>(a) / b; // 3.5
char letter = static_cast<char>('A' + 2);    // 'C'
bool valid = (5 != 0);                   // explicit meaning
// int safe{3.7}; // compile-time error: narrowing is rejected
```

Predict the output – before execution

Predicted output (the student should write this prediction before running the supplied program):

```
3 3.5
C 68
1 true
s >= u
```

Explanation: a / b performs integer division, producing 3, then converts that result to double. static_cast<double>(a) makes the division floating-point and produces 3.5. The character 'A' plus 2 is promoted to int and assigned as 'C'; printing ch + 1 prints the integer code 68. Integer 5 converts to bool true; default bool output is 1 and std::boolalpha prints true. In the final comparison, signed int -1 is converted to unsigned int, becoming a large positive value, so s < u is false and the conditional expression prints s >= u.

Alternative: explicit conversions

Writing conversions explicitly makes intent clearer and can expose narrowing at compile time when brace initialisation is used. It does not normally change asymptotic time or memory: the conversions are still constant-time operations. Explicit casts do not make an unsafe conversion safe; the programmer must still validate ranges before converting.

TASK 5 – Write Correct Expressions with the Right Precedence (4 Marks)

Fare expressions

```
bool peak = (hour >= 8 && hour < 10) ||
          (hour >= 17 && hour < 20);

long long farePaise = baseFarePaise;
if (peak) {
   farePaise += farePaise * PEAK_SURCHARGE_PERCENT / 100;
}

// Apply a concession only after its approved rate is defined.
```

The relational operators >= and < are evaluated before logical &&; && is evaluated before ||. Parentheses make the two time windows clear. The expression without the outer parentheses is equivalent because && has higher precedence than ||, but is less readable. Compound assignment += updates the left operand using the calculated right operand.

Post-increment x++ yields the old value and then increments x. Pre-increment ++x increments first and yields the new value. The expression i = i++ should be avoided because it relies on confusing sequencing behaviour and is not a meaningful way to increment a variable. Use ++i or i += 1. For integer division, -10 / 3 is -3 (truncation toward zero) and -10 % 3 is -1.

Predict the output – before execution

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Explanation: multiplication is evaluated before addition, so 5 + 3 * 2 = 11; parentheses make (5 + 3) * 2 = 16. In x++ + ++y, the old x value 5 and new y value 4 are added to produce 9; afterward x is 6 and y is 4. The conditional expression selects 6. x % 4 * 2 is evaluated left-to-right for operators of equal precedence: (6 % 4) * 2 = 4. Integer division and remainder produce 3, 1, -3 and -1. The loop adds 0 + 3 + 6 + 9 = 18.

Alternative: peak-hour lookup table

A bool peak[24] table can store true for hours 8 and 9 and 17, 18 and 19. A lookup peak[hour] takes O(1), just like the Boolean expression, but requires 24 bool elements (typically 24 bytes, though sizeof(bool) should be checked). The table is convenient when many complex time windows exist; the expression uses less data and directly communicates the two simple intervals.

TASK 6 – Design the Control Structures (4 Marks)

Pseudocode

```
START
  Read zone, hour, minute, card balance
  Validate zone and time
  IF input is invalid
     Display invalid journey and stop processing this journey
```

```
  END IF

  Use SWITCH(zone)
     Inner  : baseFare = 1000 paise
     Middle : baseFare = 2000 paise
     Outer  : baseFare = 3000 paise
     DEFAULT: reject unknown zone
  END SWITCH

  peak = (08:00 <= time < 10:00) OR
         (17:00 <= time < 20:00)
  fare = baseFare
  IF peak
     fare = fare + (fare * 20 / 100)
  END IF

  IF balance - fare >= 5000
     balance = balance - fare
     Print fare and balance
  ELSE
     Print low-balance denial
  END IF
STOP
```

Use switch for selecting one of the named zones, if–else for range checks and journey approval, and the conditional operator only for short value selection. A for loop is suitable when the number of journeys is known; a range-based for loop is clean for an existing collection of trips; a while loop is useful when reading journeys until a stop condition. continue skips an invalid journey and break exits a loop when processing must stop. If no switch case matches, default handles the invalid zone.

Complete the code – working solution

```
#include <iomanip>
#include <iostream>
#include <stdexcept>

enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak) {
   long long base = 0;
   switch (z) {
      case Zone::Inner:  base = BASE_PAISE[0]; break;
      case Zone::Middle: base = BASE_PAISE[1]; break;
      case Zone::Outer:  base = BASE_PAISE[2]; break;
      default: throw std::invalid_argument("Unknown zone");
   }
   if (peak) {
      base += base * PEAK_SURCHARGE_PERCENT / 100;
```

```
    }
    return base;
}

bool canTravel(long long balance, long long f) {
    return f >= 0 && balance >= f &&
           balance - f >= MIN_BALANCE_PAISE;
}

int main() {
    long long balance = 10000;
    struct Trip { Zone zone; bool peak; };
    Trip trips[] = {
        {Zone::Inner, false},
        {Zone::Outer, true},
        {Zone::Middle, false}
    };

    for (const Trip& t : trips) {
        const long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f)) {
            balance -= f;
            std::cout << "Fare Rs " << f / 100 << '.'
                      << std::setw(2) << std::setfill('0')
                      << f % 100 << std::setfill(' ');
        } else {
            std::cout << "Denied (low balance)";
        }
        std::cout << " balance Rs " << balance / 100 << '\\n';
    }
}
```

Expected output:

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

At exactly 10:00 AM and 8:00 PM the journey is off-peak because the end of each interval is exclusive. A helper function should use hour and minute, not hour alone, if times other than whole hours are accepted.

Alternative: array lookup versus switch

A constexpr fare array indexed by a validated zone index gives O(1) lookup, as does a switch. The array occupies storage for three long long values (typically 24 bytes); a switch may compile to branches or a jump table. Adding Airport requires extending the enum, array and validation. A switch requires a new case and an array requires another entry; a data-driven table scales more naturally when zones change often.

TASK 7 – From Procedural to Object-Oriented (4 Marks)

Procedural version

```
bool canTravelProcedural(long long balance, long long fare) {
    return fare >= 0 && balance - fare >= 5000;
}
bool deductProcedural(long long& balance, long long fare) {
    if (!canTravelProcedural(balance, fare)) return false;
    balance -= fare;
    return true;
}
```

Object-oriented version

```
class SmartCard {
private:
    long long balancePaise;
    static constexpr long long MIN_BALANCE_PAISE = 5000;

public:
    explicit SmartCard(long long initialBalance)
        : balancePaise(initialBalance) {}

    long long balance() const { return balancePaise; }

    bool canTravel(long long farePaise) const {
        return farePaise >= 0 && balancePaise >= farePaise &&
               balancePaise - farePaise >= MIN_BALANCE_PAISE;
    }

    bool deduct(long long farePaise) {
        if (!canTravel(farePaise)) return false;
        balancePaise -= farePaise;
        return true;
    }

    bool recharge(long long amountPaise) {
        if (amountPaise <= 0) return false;
        balancePaise += amountPaise;
        return true;
    }
};
```

The procedural version is simple and may be suitable for a small program, but every function receiving a mutable balance must follow the same validation rule. The SmartCard version keeps the balance private and only deducts after canTravel() succeeds. This prevents other parts of the program from directly assigning a negative balance. If the minimum balance changes, the class constant is updated in one place.

Alternative: public balance

A public balance member does not change the time or space complexity. It does remove the protection provided by encapsulation: any part of the program could write card.balancePaise = -100 or deduct

without checking the Rs 50 minimum. The private member and validated methods preserve the class invariant.

TASK 8 – Read and Print Data Safely (4 Marks)

Safe numeric input

```
#include <iostream>
#include <limits>

int readInteger(const char* prompt) {
    int value{};
    while (true) {
        std::cout << prompt;
        if (std::cin >> value) return value;

        std::cout << "Invalid input. Enter a whole number.\\n";
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\\n');
    }
}
```

When the user types letters where an integer is expected, cin enters a failed state. cin.fail() can detect the failure. cin.clear() resets the state flags, and cin.ignore() discards the invalid characters through the end of the line. Without clearing the state, later reads also fail.

Money formatting

```
#include <iomanip>
#include <iostream>

void printMoney(long long paise) {
    std::cout << "Rs " << paise / 100 << '.'
              << std::setw(2) << std::setfill('0')
              << paise % 100 << std::setfill(' ');
}
```

For floating-point display, fixed and setprecision(2) print two decimal places. For integer paise, division and remainder give exact rupees and paise, and setw(2)/setfill('0') preserve a leading zero. A journey report should show journey number, zone, time, peak/off-peak status, fare, decision and remaining balance.

Alternative: getline and stoi

With getline(), the program reads the complete input line and converts it using std::stoi inside try/catch. Both approaches take O(L) time for a line of length L. getline uses a string buffer proportional to the line length. cin >> is convenient for token-based input; getline gives control over the entire line and exception handling makes invalid text easier to report. Care is needed to reject trailing characters after the number.

TASK 9 – Implement and Test the Fare Logic (4 Marks)

Test cases

Test	Input / condition	Expected result	Reason
T1	Inner, 07:59, balance Rs 100	Fare Rs 10; balance Rs 90	Off-peak Inner.
T2	Middle, 08:00, balance Rs 100	Fare Rs 24; balance Rs 76	Peak surcharge applies.
T3	Outer, 10:00, balance Rs 100	Fare Rs 30; balance Rs 70	10:00 is off-peak.
T4	Outer, 17:00, balance Rs 100	Fare Rs 36; balance Rs 64	Peak surcharge applies.
T5	Outer, 20:00, balance Rs 100	Fare Rs 30; balance Rs 70	20:00 is off-peak.
T6	Inner fare Rs 10, initial balance Rs 60	Allowed; remaining Rs 50	Exactly minimum balance remains.
T7	Unknown zone code	Reject journey	Zone is outside valid enum/input range.
T8	Negative recharge	Reject recharge	Recharge must be positive.
T9	Letters entered for hour	Show input error and recover	cin failure is cleared and bad line discarded.

Five-journey hand calculation example: start with Rs 200. Journey 1 Inner off-peak = Rs 10; Journey 2 Middle peak = Rs 24; Journey 3 Outer off-peak = Rs 30; Journey 4 Inner peak = Rs 12; Journey 5 Middle off-peak = Rs 20. Total = Rs 96 and remaining balance = Rs 104. All five journeys are allowed because the balance remains at least Rs 50 after each deduction.

Alternative: table-driven tests

A table of input cases and expected fares can be processed in a loop. For t test cases, execution time is O(t), and memory is O(t) for the test table. Adding a new boundary case means adding one row instead of writing another independent test block. The table also makes expected and actual results easy to compare.

TASK 10 – Compare Expectations with Execution (4 Marks)

Network-scale total analysis

For an illustrative upper-bound calculation, assume every journey is charged Rs 36. This is the Outer-zone peak fare. The exact total is therefore the number of journeys multiplied by 3,600 paise.

Journeys/day	Total in rupees	Total in paise	32-bit signed int safe in paise?
20,000	Rs 720,000	72,000,000	Yes
500,000	Rs 18,000,000	1,800,000,000	Yes, below 2,147,483,647
3,000,000	Rs 108,000,000	10,800,000,000	No

This is a stress-case calculation, not a measured actual collection; real totals depend on the zone mix, peak share and concessions. A signed 32-bit int can overflow when storing the whole-network total in paise. long long is appropriate. double has a wide range but is not the preferred exact money representation.

Compiler warnings and measurements

Compile with -Wall -Wextra -pedantic. Record every warning from your own build and explain the line and reason. Common warnings include signed/unsigned comparisons, unused variables, narrowing or conversion warnings (when -Wconversion is also enabled), and missing return paths. Do not write that a warning occurred unless it appeared in your compiler output.

Alternative: accumulator types

Type	Behaviour at 3,000,000 × 3,600 paise	Time / storage
int (commonly 32-bit)	Overflows; signed overflow is undefined behaviour in C++.	O(1) per addition; commonly 4 bytes.
long long	Stores 10,800,000,000 exactly.	O(1) per addition; commonly 8 bytes.
double	Can represent the magnitude, but decimal currency may not be exact.	O(1) per addition; commonly 8 bytes.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

The recommended original design uses enum class Zone, constexpr fare and surcharge constants, long long integer paise, validated time input, a FareTable or fare() function, and a SmartCard class with a private balance. A Journey record stores the zone and time, while a loop processes each trip. These choices directly address the original defects: repeated fare literals, floating-point paisa errors, inconsistent time boundaries and negative balances.

- Fare revision: update the central constexpr fare table and surcharge policy, then rebuild and rerun boundary tests.
- Precision: integer paise avoids floating-point currency rounding.
- Safety: SmartCard checks the Rs 50 minimum before deduction and does not mutate balance when a journey is denied.
- Scalability: long long supports the estimated whole-network total in paise.

- Limitations: student/senior concession rates are not specified in the brief; persistence, concurrency, refunds, audit logs and secure card authentication are outside this small program.
- Whole-network correctness: O(1) fare calculation and balance validation per journey are suitable algorithmically, but production deployment also requires database consistency, thread safety, monitoring and transaction logging.

Alternative: configuration file

If fares are loaded from a configuration file at gate startup, the program can update prices without recompilation. Reading z fare values takes O(z) startup time and O(z) storage. This adds file validation, error handling, versioning and secure configuration management. A compiled constexpr table is simpler and safer for a fixed policy; a configuration file is more flexible when fare changes are frequent and operationally controlled.

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

A. Impact analysis

The new requirement adds Airport at Rs 60, a 10% weekend discount that cannot combine with the peak surcharge, and a monthly pass for unlimited journeys in two selected zones. The Zone enum gains Airport. The fare table gains a fourth entry. The switch statement must handle Airport or be replaced by validated table lookup. Fare calculation must accept day type and discount policy. SmartCard must distinguish stored-value journeys from pass-authorised journeys, and a new MonthlyPass class is needed.

B. Evaluation of original design

The switch-based design can absorb Airport by adding a new case, but repeated switch logic becomes harder to maintain as zones and fare policies increase. SmartCard can still protect stored balance, but it should not deduct a fare when a valid monthly pass covers the selected zone. The original fare() signature fare(Zone, bool peak) is no longer sufficient because it does not know whether the journey is on a weekend or whether a pass applies.

Worked examples: Airport weekday peak fare = Rs 60 + 20% = Rs 72. Airport weekend off-peak fare = Rs 60 − 10% = Rs 54. Airport weekend peak fare is also Rs 54 because the weekend discount does not combine with the peak surcharge; the weekend rule takes precedence. A valid monthly pass covering Airport authorises the journey at Rs 0 fare for that journey.

C. Adapted design and implementation

The revised design uses a fare table indexed by a validated Zone value, a separate weekend/peak policy, and a MonthlyPass class containing the two covered zones and expiry month. The code below demonstrates the fare-policy and pass-coverage logic. A production system would additionally validate dates, pass ownership, expiry and transaction records.

```
#include <array>
#include <iostream>
#include <stdexcept>

enum class Zone : int { Inner = 0, Middle = 1, Outer = 2, Airport = 3 };

constexpr std::array<long long, 4> BASE_FARE_PAISE{
   1000, 2000, 3000, 6000
};
```

```
long long calculateFare(Zone zone, bool peak, bool weekend) {
    const int index = static_cast<int>(zone);
    if (index < 0 || index >= static_cast<int>(BASE_FARE_PAISE.size())) {
        throw std::invalid_argument("Invalid zone");
    }

    long long fare = BASE_FARE_PAISE[static_cast<std::size_t>(index)];

    // Weekend discount has priority; it never combines with peak surcharge.
    if (weekend) {
        fare -= fare * 10 / 100;
    } else if (peak) {
        fare += fare * 20 / 100;
    }
    return fare;
}

class MonthlyPass {
private:
    Zone firstZone;
    Zone secondZone;
    int validMonth; // Demonstration: 1..12

public:
    MonthlyPass(Zone a, Zone b, int month)
        : firstZone(a), secondZone(b), validMonth(month) {}

    bool covers(Zone zone, int travelMonth) const {
        return travelMonth == validMonth &&
               (zone == firstZone || zone == secondZone);
    }
};

int main() {
    std::cout << "Airport weekday peak Rs "
              << calculateFare(Zone::Airport, true, false) / 100.0
              << '\\n';
    std::cout << "Airport weekend fare Rs "
              << calculateFare(Zone::Airport, false, true) / 100.0
              << '\\n';

    MonthlyPass pass(Zone::Inner, Zone::Airport, 11);
    std::cout << "Airport covered by November pass: "
              << (pass.covers(Zone::Airport, 11) ? "Yes" : "No")
              << '\\n';
}
```

For financial output, the demonstration should ideally use the same integer-paise formatter used earlier instead of dividing by 100.0; floating-point formatting is shown here only to keep the short demonstration readable. The fare calculation itself remains integer arithmetic.

D. Evidence-based defence

Test	Expected result	Why it validates the revision
Airport weekday peak	Rs 72.00	New fourth zone and peak surcharge.
Airport weekend off-peak	Rs 54.00	10% weekend discount.
Airport weekend peak	Rs 54.00	Weekend discount takes precedence; no stacking.
Monthly pass covers Airport in valid month	Journey accepted; fare Rs 0	Pass covers one of the two selected zones.
Monthly pass does not cover Outer	Normal fare calculated	Uncovered zone uses stored-value fare.
Pass month expired	Pass rejected	Expiry month must match travel month.

Decision: MODIFY the original design rather than completely redesign it. Retain the SmartCard encapsulation and integer-paise arithmetic, but replace the small switch-based fare selection with a four-entry fare table and separate the fare policy from payment authorisation. Add MonthlyPass as a distinct class. This is a balanced change: the existing reliable balance protection is retained, while new fare rules are added without placing every policy inside SmartCard.

Overall Conclusion

The metro fare system should use object-oriented classes to keep data and behaviour together, enum class for safe zone values, constexpr for fixed policy constants, and long long integer paise for exact money calculations. Clear peak-hour boundaries, input validation and a minimum-balance check prevent the original fare and balance errors. Testing at boundary times and at network scale demonstrates why data types and control structures matter. When Airport, weekend discounts and monthly passes are introduced, the design should be modified with a fare table, a separate discount policy and a MonthlyPass class. This design remains easy to extend while preserving the SmartCard balance invariant.

References

- ISO/IEC 14882:2020. Programming Languages – C++ (C++20) – Standard.
- cppreference.com. C++ language reference: fundamental types, conversions, enum declarations, classes and operator precedence. https://en.cppreference.com/
- GNU Project. GCC online documentation: warning options and C++ compiler options. https://gcc.gnu.org/onlinedocs/
- Stroustrup, B. The C++ Programming Language. 4th Edition, Addison-Wesley, 2013.

Final Submission Checklist

- ☐ Student name, register number, date and working GitHub URL completed on cover page.
- ☐ All Task01–Task12 folders created in the repository.
- ☐ Every required C++ program saved, compiled and executed individually.
- ☐ Complete the Code programs include the completed TODO sections and output screenshots.
- ☐ Predicted outputs were written before execution and compared with actual outputs.
- ☐ Compiler name/version, exact compile command, warnings and sizeof results recorded.
- ☐ UML class diagram, pseudocode, test tables and complexity analyses included.
- ☐ README.md contains run instructions and result notes for every task.
- ☐ At least one progressive commit made for every task.
- ☐ All placeholders and sample evidence replaced with the student's own execution evidence.

Important: Do not submit the sample GitHub URL as if it were your real repository. Create the repository, upload your own executed code and screenshots, then paste the actual working link.